

Building AI Agents

1. Introduction to LangChain

What is LangChain?

LangChain is a framework for building AI-powered applications and agents.

It helps developers connect:

- LLMs (GPT, Gemini, Claude)
- APIs
- Databases
- Memory
- Tools

Architecture

```
User
  ↓
Agent
  ↓
LLM
  ↓
Tools / APIs / Database
  ↓
Response
```

Daily Life Example

Personal Assistant

User:

What's on my schedule today?

Agent:

1. Reads calendar
 2. Checks meetings
 3. Generates summary
-

Installation

```
pip install langchain
pip install langchain-openai
```

2. Core Concepts

A. Chains

A chain connects multiple steps together.

Daily Life Problem

Resume Analyzer

```
Resume
  ↓
Extract Skills
  ↓
Generate Feedback
  ↓
Suggest Improvements
```

Code

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["skill"],
    template="Explain {skill} in simple words."
)

print(template.format(skill="Python"))
```

Output:

```
Explain Python in simple words.
```

B. Prompts

Prompts tell the model what to do.

Bad Prompt

```
Tell me about databases.
```

Good Prompt

```
Explain databases in 5 points for beginners.
```

Code

```
prompt = """
You are a teacher.

Explain SQL databases in simple terms.
"""

print(prompt)
```

3. Tool Integration

Tools allow agents to interact with the outside world.

A. External APIs

Daily Life Problem

Before traveling, check weather.

Code

```
import requests

city = "Delhi"

# Example API call
print(f"Getting weather for {city}")
```

Agent Workflow

```
User asks
      ↓
Agent
      ↓
Weather API
      ↓
Travel Advice
```

Real-Life Example

User:
Should I carry an umbrella?

Agent:

Weather API says rain.

Carry umbrella.

B. Databases

Agents can read stored information.

Daily Life Problem

Student Management System

Agent checks:

Student ID
Attendance
Marks

SQLite Example

```
import sqlite3

conn = sqlite3.connect("students.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students(
id INTEGER,
name TEXT
)
""")

cursor.execute(
"INSERT INTO students VALUES(1, 'Rahul') "
)

conn.commit()

for row in cursor.execute(
"SELECT * FROM students"
):
    print(row)
```

4. Memory Integration

Memory allows agents to remember information.

Without Memory

User:
My name is Rahul.

User:
What's my name?

Agent:
I don't know.

With Memory

Code

```
memory = {}

memory["name"] = "Rahul"

print(
    f"Hello {memory['name']}"
)
```

Output:

Hello Rahul

Daily Life Example

Food Delivery App

Agent remembers:

```
Favorite restaurant
Preferred payment method
Delivery address
```

5. Agent Design

Task-Specific Agents

Each agent specializes in one task.

Example 1: Study Agent

```
def study_agent(topic):
    return f"Explaining {topic}"

print(study_agent("Python"))
```

Example 2: Fitness Agent

```
def fitness_agent(goal):  
    return f"Workout plan for {goal}"  
  
print(  
    fitness_agent("Weight Loss")  
)
```

Daily Life Problem

Student needs:

Notes
Quiz
Summary

A Study Agent handles only education-related tasks.

6. Reasoning Systems

Multi-Step Problem Solving

Agents solve complex tasks by breaking them into smaller steps.

Daily Life Problem

Planning a Trip

Steps:

```
Check weather  
↓  
Find flights  
↓  
Book hotel  
↓  
Calculate budget
```

Code

```
goal = "Trip Planning"  
  
steps = [  
    "Check weather",
```

```
    "Find flights",
    "Book hotel",
    "Calculate budget"
]

for step in steps:
    print(step)
```

Reasoning Example

Problem:

Should I go to office?

Reasoning:

```
weather = "Rain"

if weather == "Rain":
    print("Take cab")
else:
    print("Use bike")
```

Output:

Take cab

7. Debugging Agents

Sometimes agents make mistakes.

Common Issues

1. Wrong Tool Selection

Agent:

Need weather information

Uses:

Calculator Tool

Wrong choice.

2. Missing Memory

User:

My name is Rahul.

Later:

What's my name?

Agent forgets.

3. Invalid API Responses

```
try:
    response = api_call()
except Exception:
    print("API failed")
```

Debugging Tips

Log Agent Steps

```
steps = [
    "Get weather",
    "Analyze traffic",
    "Recommend transport"
]

for step in steps:
    print("Executing:", step)
```

Output:

```
Executing: Get weather
Executing: Analyze traffic
Executing: Recommend transport
```

8. Mini Project: Task Automation Agent

Problem

A working professional wants help managing daily tasks.

Agent should:

Prioritize tasks

Suggest transport

Send reminders

Step 1: Create Tasks

```
tasks = [  
    "Team Meeting",  
    "Submit Report",  
    "Gym"  
]
```

Step 2: Planning

```
print("Today's Tasks")  
  
for task in tasks:  
    print("-", task)
```

Step 3: Reasoning

```
weather = "Rain"  
  
if weather == "Rain":  
    transport = "Cab"  
else:  
    transport = "Bike"
```

Step 4: Execution

```
print(  
f"Recommended Transport: {transport}"  
)
```

Complete Code

```
tasks = [  
    "Team Meeting",  
    "Submit Report",  
    "Gym"  
]  
  
weather = "Rain"  
  
print("Today's Tasks")  
  
for task in tasks:
```

```
        print("-", task)

if weather == "Rain":
    transport = "Cab"
else:
    transport = "Bike"

print(
    "\nRecommended Transport:",
    transport
)

print(
    "Reminder: Team Meeting at 10 AM"
)
```

Output

Today's Tasks

- Team Meeting
- Submit Report
- Gym

Recommended Transport: Cab

Reminder:
Team Meeting at 10 AM

Real-Life AI Agent Example

Smart Office Assistant

User:

Prepare my workday.

Agent performs:

1. Read Calendar
2. Check Weather
3. Analyze Traffic
4. Prioritize Tasks
5. Send Summary

Workflow

```
User Goal
  ↓
Agent
  ↓
Planning
  ↓
```

Reasoning
↓
Tool Usage
↓
Execution
↓
Result

LangChain Agent Example

```
from langchain.agents import initialize_agent
from langchain.agents import AgentType
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key="YOUR_API_KEY"
)

agent = initialize_agent(
    tools=[],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION
)

response = agent.run(
    "Explain Python in simple words"
)

print(response)
```

Quick Summary

Topic	Purpose	Daily Life Example
LangChain	Build AI applications	Personal assistant
Chains	Connect tasks	Resume analyzer
Prompts	Guide AI	Teaching assistant
APIs	External information	Weather checking
Databases	Store data	Student records
Memory	Remember users	Food delivery preferences
Task-Specific Agents	Specialized tasks	Study agent
Reasoning	Multi-step thinking	Trip planning
Debugging	Fix agent errors	Missing memory
Automation Agent	Perform tasks automatically	Daily office assistant

Key Difference

LLM Application

User → LLM → Response

AI Agent

User → Agent

